System Design Interview Handbook

High-Level Design (HLD) of 12 Classic Systems
with Architecture Diagrams, Scale Estimates & Trade-offs

Jaydip

Focus	HLD interview rounds (SDE-2 / Senior)
Format	Problem → Requirements → Architecture → Deep dives

Contents

1. How to Answer Any System Design Question (Framework)
2. Core Building Blocks & Back-of-the-Envelope Numbers
3. URL Shortener (TinyURL / bit.ly)
4. Rate Limiter
5. Chat System (WhatsApp / Messenger)
6. News Feed (Instagram / Twitter / Facebook)
7. Video Streaming (YouTube / Netflix / Hotstar)
8. Ride Sharing (Uber / Ola)
9. Web Crawler (Google-scale)
10. Notification System (Push / Email / SMS)
11. Distributed Key-Value Cache (Redis / Memcached-like)
12. Payment System (Stripe / Razorpay-like)
13. Search Autocomplete (Typeahead)
14. Distributed Job Scheduler (Cron-as-a-Service)
15. Rapid-Fire: Trade-offs Interviewers Love to Probe

Jaydip

1. How to Answer Any System Design Question

Interviewers are evaluating your **structured thinking**, not a memorized architecture. Use this 4-step framework in every HLD round (budget ~45 min):

Step 1 — Clarify requirements (5 min)

- **Functional:** What exactly should the system do? Pick 2–3 core features; explicitly de-scope the rest.
- **Non-functional:** Scale (DAU, QPS), latency targets, availability vs consistency, durability, read/write ratio.
- Ask: 'Is this read-heavy or write-heavy?' — it drives almost every later decision.

Step 2 — Back-of-the-envelope estimation (5 min)

- Estimate QPS (avg and peak ≈ 2–5× avg), storage per year, bandwidth, and cache size (80/20 rule: cache 20% of hot data).

Step 3 — High-level design (15–20 min)

- Draw: clients → LB/API gateway → services → cache → DB → queue → workers. Define the API (3–4 endpoints) and the core data model first.
- Walk through the two most important flows end-to-end (e.g., one write path, one read path).

Step 4 — Deep dive & wrap-up (15 min)

- Let the interviewer pick a component, or propose one yourself: sharding strategy, cache invalidation, hot keys, failure handling.
- Close with bottlenecks, single points of failure, monitoring, and what you would improve with more time.

Golden rules: state assumptions out loud, always give a trade-off when you make a choice ('I choose X over Y because...'), and never gold-plate — start simple, then scale.

Jaydip

2. Core Building Blocks & Numbers to Memorize

Building blocks

- **Load balancer:** L4 (TCP, fast) vs L7 (HTTP, smart routing). Algorithms: round robin, least connections, consistent hashing.
- **Caching:** cache-aside (most common), write-through, write-back. Eviction: LRU/LFU. Problems: hot keys, thundering herd (fix: request coalescing, jittered TTLs).
- **Databases:** SQL for transactions/joins; NoSQL (Cassandra, DynamoDB) for write-heavy scale; scale via replication (read replicas) then sharding (hash / range / geo).
- **Message queues (Kafka, SQS, RabbitMQ):** decouple producers/consumers, absorb spikes, enable async work and retries with dead-letter queues.
- **CDN:** serve static/media content from edge locations; push vs pull CDN.
- **CAP theorem:** under a network partition choose Consistency (CP: banking) or Availability (AP: feeds, likes). Most web systems choose AP + eventual consistency.
- **Consistent hashing:** add/remove nodes while remapping only ~1/N keys; virtual nodes fix uneven load.
- **Idempotency:** retries are everywhere, so writes need idempotency keys — a favourite interviewer probe.

Latency numbers (order of magnitude)

Operation	Approx. time
L1 cache reference	0.5 ns
Main memory reference	100 ns
Read 1 MB sequentially from memory	0.25 ms
SSD random read	~150 µs
Read 1 MB sequentially from SSD	~1 ms
Round trip within same datacenter	~0.5 ms
Disk seek (HDD)	~10 ms
Round trip cross-continent (e.g., India ↔ US)	~150 ms

Handy estimation shortcuts

- 1 day ≈ 86,400 s ≈ 10^5 s. So 100M requests/day ≈ 1,150 QPS average; assume peak ≈ 2–5×.
- 1M users × 1 KB each = 1 GB. 1M users × 1 MB each = 1 TB.
- A single modern server: ~10K–50K simple QPS; a single MySQL primary: ~1–5K writes/s; Redis: ~100K ops/s.

3. URL Shortener (TinyURL)

Problem: Given a long URL, return a short unique alias; redirect users who open the alias. The 'hello world' of system design — almost guaranteed as a warm-up.

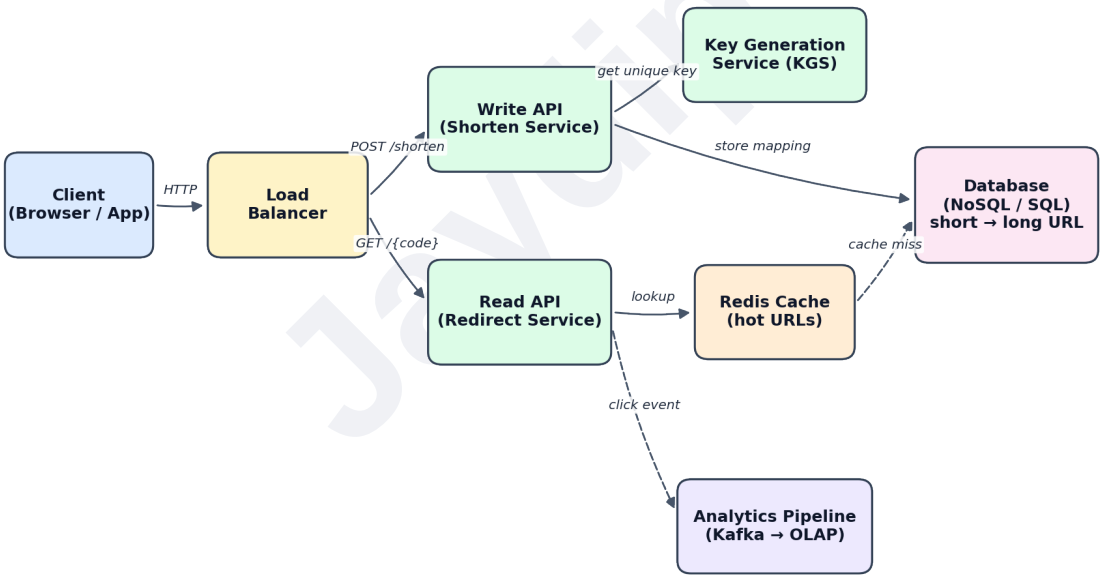
Requirements

- **Functional:** shorten(longURL) → shortURL; redirect(shortURL) → 301/302 to longURL; optional custom alias, expiry, click analytics.
- **Non-functional:** Very read-heavy (~100:1 read:write), low latency redirects (<50 ms), high availability (AP system), short codes must never collide.

Back-of-the-envelope

- Writes: 100M new URLs/month ≈ 40/s. Reads: 100:1 → ~4,000 QPS redirects.
- Storage: 100M/month × 500 bytes × 5 years ≈ 3 TB — easily fits a modest DB cluster.
- Code length: base62 (a-z, A-Z, 0-9). 62^7 ≈ 3.5 trillion — 7 characters is plenty.

Architecture



URL shortener: separate write (shorten) and read (redirect) paths; KGS avoids collisions; cache serves hot redirects.

How it works

- **Write path:** API takes long URL → gets a pre-generated unique key from the Key Generation Service (KGS) → stores {shortCode → longURL} in DB → returns short URL.
- **Read path:** GET /{code} → check Redis → on miss read DB and backfill cache → respond 302 (allows analytics) or 301 (cacheable, fewer hits to you).
- KGS pre-generates keys offline into a 'available keys' pool, handing them out atomically — no runtime collision checks needed.
- Click events are fired async to Kafka for analytics so redirects stay fast.

Key design decisions & trade-offs (deep-dive material)

- **Hash vs counter vs KGS:** MD5+truncate risks collisions; auto-increment counter leaks volume and is a single point of failure (fix: ranged counters via ZooKeeper); KGS is the cleanest answer.
- **301 vs 302:** 301 is cached by browsers (less load, no analytics); 302 hits your server every time (analytics, mutable destinations).
- **DB choice:** simple key-value access pattern → DynamoDB/Cassandra scale horizontally with no joins needed.
- Expired links: lazy deletion on read + periodic cleanup job, rather than active scanning.

Jaydip

4. Rate Limiter

Problem: Limit how many requests a client (user, IP, API key) can make in a time window, e.g., 100 requests/min, to prevent abuse, DoS and cost overruns.

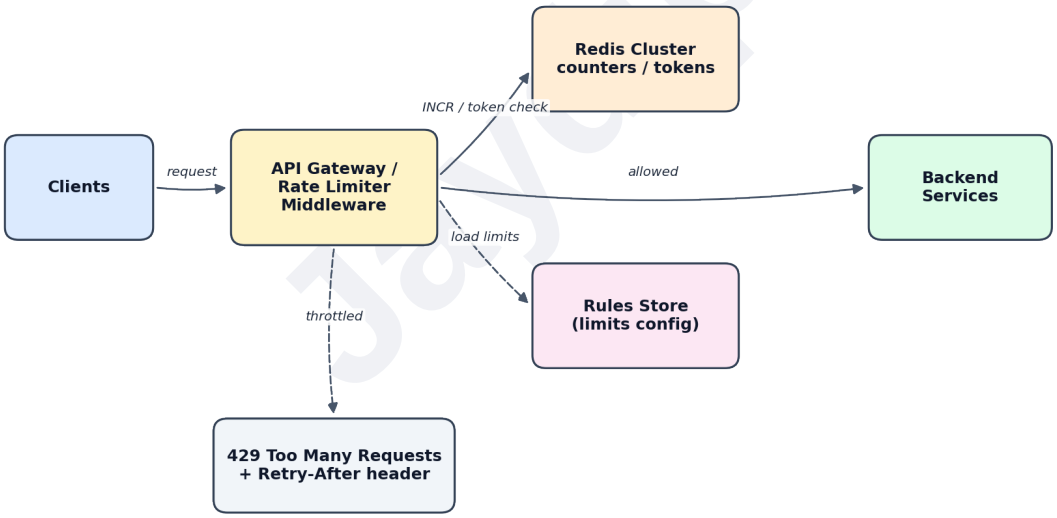
Requirements

- **Functional:** Accurately limit per-client rules; return HTTP 429 with Retry-After; support multiple rules (per-user, per-endpoint).
- **Non-functional:** Ultra-low added latency (<1–2 ms), works across a distributed fleet, fail-open vs fail-closed behaviour must be a conscious choice, minimal memory.

Back-of-the-envelope

- 10M users, tracking one counter each ≈ tens of MB in Redis — cheap.
- Rate-limit check adds one Redis round trip (~0.5 ms in-DC) per request; use Lua scripts to make check-and-increment atomic.

Architecture



Rate limiter as gateway middleware; counters live in Redis so all gateway instances share state.

How it works

- Client request hits the API gateway; middleware computes the key (e.g., user_id:endpoint:window) and runs an atomic Redis operation.
- If under the limit → forward to backend; else → 429 with Retry-After and X-RateLimit-* headers.
- Rules (limits per plan/endpoint) are stored centrally and cached in each gateway worker.

Key design decisions & trade-offs (deep-dive material)

- **Algorithms:** Token bucket (allows bursts; industry default — state = tokens + last-refill timestamp), Leaky bucket (smooth outflow), Fixed window (2× burst at boundary), Sliding window log (exact, memory-heavy), Sliding window counter (best accuracy/memory balance).

- **Distributed race conditions:** two gateways incrementing simultaneously — solve with Redis Lua scripts (atomic) rather than read-then-write.
- **Fail-open or fail-closed** if Redis is down? Fail-open for user-facing APIs (availability), fail-closed for payment/security endpoints.
- Mention local in-memory pre-limits + global Redis check to cut latency for hot clients.

Jaydip

5. Chat System (WhatsApp / Messenger)

Problem: 1:1 and group messaging with online presence, delivery/read receipts, and offline delivery. Tests your knowledge of stateful connections and fanout.

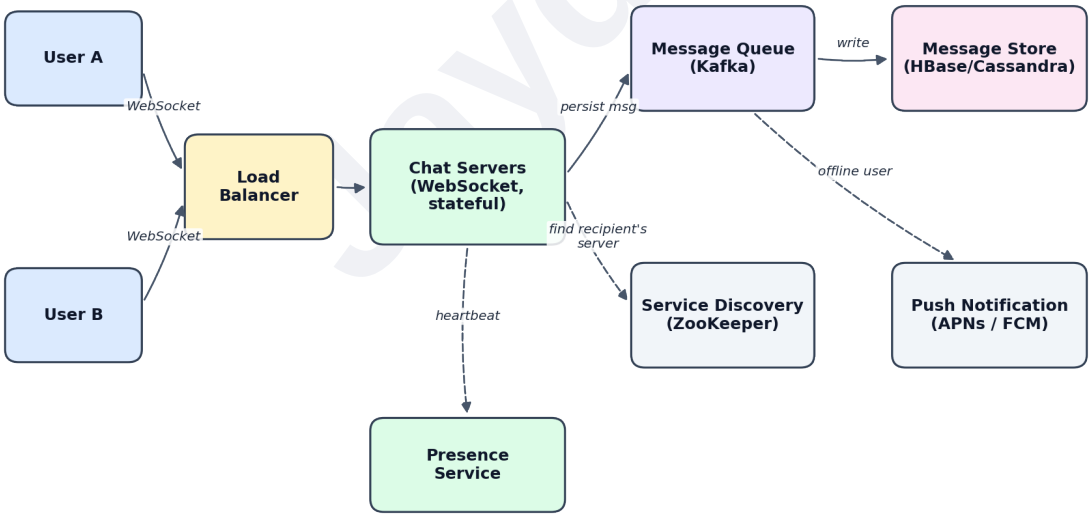
Requirements

- **Functional:** Send/receive 1:1 and group messages (group ≤ ~200 members), delivery + read receipts, presence (online/last seen), offline push notifications, multi-device sync.
- **Non-functional:** Real-time (<100 ms in-region), messages must never be lost (durable before ack), ordered per conversation, ~1B DAU scale, E2E encryption as a bonus point.

Back-of-the-envelope

- 1B DAU × 40 messages/day = 40B msgs/day ≈ 460K msgs/s average, ~1.5M/s peak.
- Persistent connections: 1B concurrent WebSockets; at ~1M connections per beefy server → ~1,000 chat servers.
- Storage: 40B × 100 bytes ≈ 4 TB/day → wide-column store (HBase/Cassandra) keyed by conversation_id.

Architecture



Chat: stateful WebSocket servers + Kafka for durability; service discovery routes messages to the recipient's server.

How it works

- Clients hold a persistent **WebSocket** to a chat server (HTTP request/response can't push server → client).
- Send flow: msg → sender's chat server → Kafka (durable, ack to sender = 'single tick') → consumer writes to message store → discovery service finds which chat server holds recipient's connection → deliver ('double tick').
- Recipient offline → message stays in store/inbox queue; push notification via APNs/FCM; delivered on reconnect (sync via per-user sequence numbers).

- Group message: fanout service writes one copy and delivers a pointer to each member's inbox (fanout-on-write for small groups).

Key design decisions & trade-offs (deep-dive material)

- **Why WebSocket over polling/long-polling:** full-duplex, one connection, lowest latency; long-polling wastes connections and adds delay.
- **Message ordering:** per-conversation sequence numbers generated by the server owning the conversation — global ordering is unnecessary and expensive.
- **Message ID generation:** Snowflake-style IDs (timestamp + machine + sequence) → sortable + unique.
- **Presence:** heartbeat every ~10 s; mark offline after ~30 s missed — avoids flapping on brief network blips.
- Chat servers are stateful → interviewers probe failover: on server death, clients reconnect via LB; discovery table (ZooKeeper/Redis) is updated.

Jaydip

6. News Feed (Instagram / Twitter)

Problem: Users follow others and see a ranked/reverse-chronological feed of posts. The classic fanout read-vs-write problem.

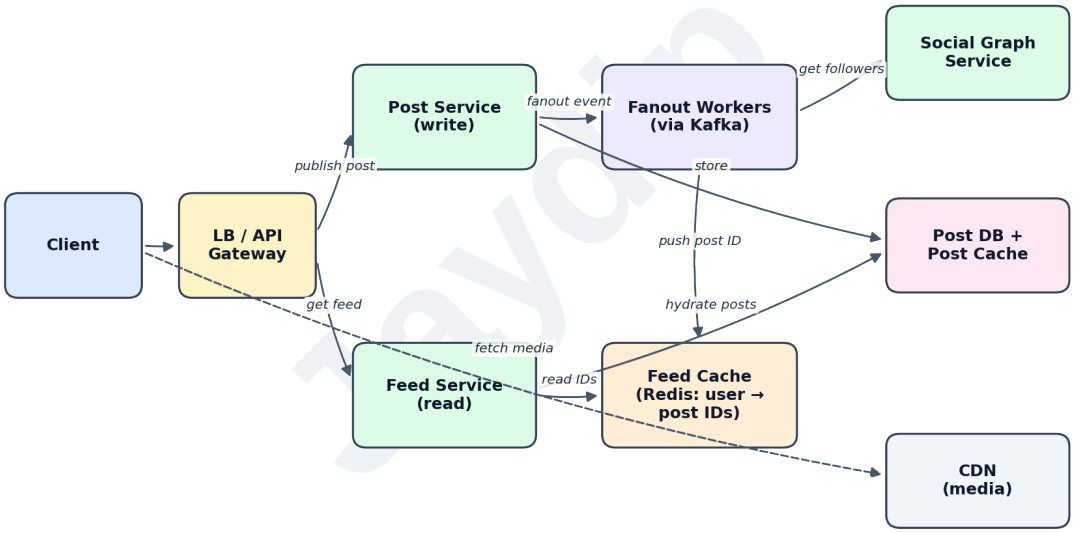
Requirements

- **Functional:** Publish post (text/media), follow/unfollow, get feed (paginated), like/comment counts.
- **Non-functional:** Feed load p99 < 200 ms, eventual consistency fine (a post may take seconds to appear), extremely read-heavy, celebrity accounts with 100M followers must not melt the system.

Back-of-the-envelope

- 300M DAU, each loads feed 10×/day → 3B feed reads/day ≈ 35K QPS avg, ~100K peak.
- Writes: 100M posts/day ≈ 1,200/s — read:write ≈ 30:1 → optimize reads.
- Feed cache: 300M users × 500 post IDs × 8 bytes ≈ ~1.2 TB of Redis — feasible across a cluster.

Architecture



News feed: fanout-on-write pushes post IDs to follower feed caches; read path just hydrates cached IDs.

How it works

- **Write path (fanout-on-write / push):** post → Post Service stores it → event to Kafka → fanout workers fetch follower list from graph service → push post ID into each follower's Redis feed list (capped, e.g., 800 IDs).
- **Read path:** Feed Service reads the user's cached ID list → hydrates from post cache/DB → merges with ranking signals → returns page; media loads from CDN.
- **Celebrity problem (fanout-on-read / pull):** don't fanout for users with >X followers; instead, at read time merge follower's precomputed feed with recent posts pulled from celebrities they follow — the **hybrid model** is the expected answer.

Key design decisions & trade-offs (deep-dive material)

- **Push vs pull vs hybrid:** push = fast reads, huge write amplification for celebrities; pull = cheap writes, slow reads; hybrid gets both — say this explicitly.
- Store only post **IDs** in feed cache, not full posts — dedupe content, shrink memory.
- Inactive users: skip fanout for users inactive > 30 days; rebuild their feed lazily on login.
- Ranking: fetch 2–3× candidates and score with a lightweight ML model at read time — mention it as an extension, don't design it.

Jaydip

7. Video Streaming (YouTube / Netflix)

Problem: Users upload videos; the system transcodes them and streams to millions of viewers on any device/network.

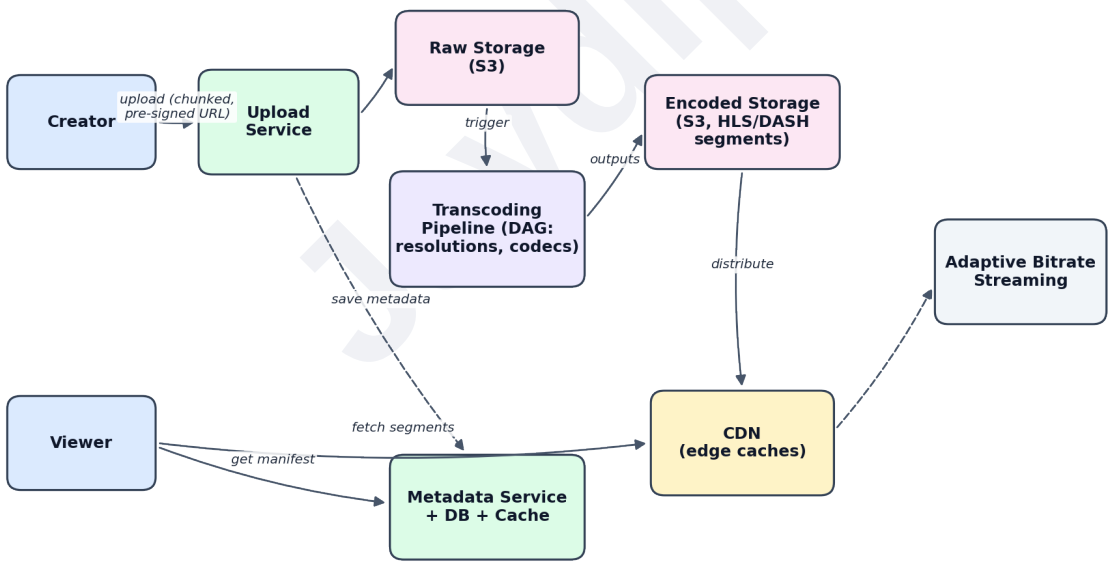
Requirements

- **Functional:** Upload video, transcode to multiple resolutions/codecs, stream with adaptive bitrate, thumbnails/metadata, view counts.
- **Non-functional:** Smooth playback on flaky networks (ABR), high durability for originals, upload can be slow but must be resumable, massive read fanout (a viral video = millions of concurrent viewers).

Back-of-the-envelope

- 5M uploads/day × 300 MB avg = 1.5 PB/day raw; transcoded copies ≈ +1× → object storage + lifecycle tiers.
- 1B watch-hours/day; ~95% of egress must come from CDN or costs explode — CDN is the real hero.
- Transcoding a 10-min video ≈ minutes of compute → must be async, parallelized by splitting video into chunks (GOPs).

Architecture



Video pipeline: upload to object storage, DAG-based transcoding, HLS/DASH segments pushed to CDN; clients adapt bitrate per segment.

How it works

- **Upload:** client gets a pre-signed URL → uploads in chunks directly to S3 (resumable, bypasses app servers) → completion event triggers the pipeline.
- **Transcoding DAG:** split video into chunks → parallel workers encode each chunk into a ladder of resolutions (240p…4K) and codecs (H.264/VP9/AV1) → stitch → package as HLS/DASH segments (2–10 s each) + manifest.
- **Streaming:** player fetches the manifest, then pulls segments from the nearest CDN edge; it measures throughput per segment and switches quality up/down — that is adaptive bitrate (ABR).

- View counts: raw events → Kafka → streaming aggregation (approximate counts OK) — never increment a DB row per view.

Key design decisions & trade-offs (deep-dive material)

- **Why segments + manifest instead of one file:** enables ABR, seeking, CDN caching, and parallel transcode.
- **Pre-signed URL upload** keeps petabytes off your app servers — a detail interviewers reward.
- Popular vs long-tail: push hot content to CDN proactively; long-tail pulled on demand (pull CDN).
- Cost lever: cold originals → Glacier-style storage; encode most-watched videos with better (more expensive) codecs first.

Jaydip

8. Ride Sharing (Uber)

Problem: Match riders with nearby drivers in real time, track the trip, and compute fares. Tests geospatial indexing and high-frequency location writes.

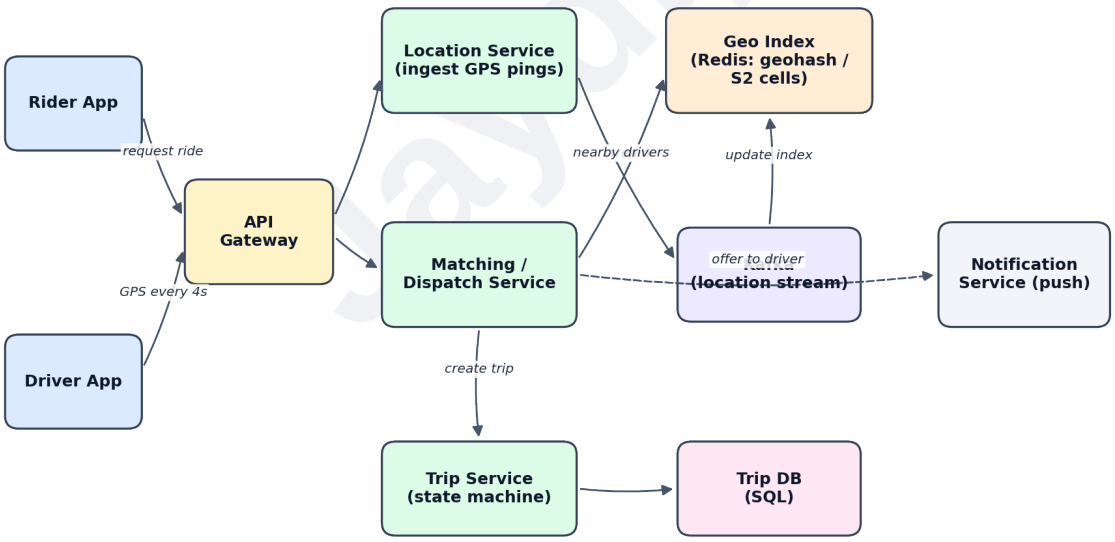
Requirements

- **Functional:** Rider requests ride with pickup/destination; find nearby drivers and dispatch; live driver tracking on map; trip lifecycle (accept → pickup → complete) and fare.
- **Non-functional:** Matching in < a few seconds, location updates every ~4 s per driver, high write throughput on locations, trip/payment data strongly consistent, location data can be slightly stale.

Back-of-the-envelope

- 5M active drivers × 1 ping/4 s = 1.25M location writes/s — clearly not for a relational DB; keep it in memory.
- Only the *latest* location matters for matching → store current position in Redis geo index; history goes to Kafka → cold store.
- 1M rides/hour peak ≈ 280 dispatches/s — trivial compared to the location firehose.

Architecture



Ride sharing: location firehose flows through Kafka into a geo index; matching service queries nearby cells and dispatches.

How it works

- Driver apps stream GPS via the gateway → Location Service → Kafka → geo index updated (driver_id → geohash/S2 cell).
- Rider requests a ride → Matching Service queries the rider's cell + neighboring cells for available drivers → ranks by ETA (road distance, not straight-line) → offers to top driver with a short TTL, cascading to the next on timeout.
- Trip Service owns the trip state machine (requested → matched → arriving → in-trip → completed) in a transactional DB; notifications go out via push.

- Fare: distance/time accumulated from the location stream + surge multiplier computed per cell from supply/demand.

Key design decisions & trade-offs (deep-dive material)

- **Geospatial index choices:** geohash (simple, prefix search, cell-boundary edge cases → search neighbors too), S2/H3 cells (Uber uses H3), quadtree (adaptive density). Any is fine — explain neighbor-cell queries.
- **Why Redis for live locations:** ephemeral, high write rate, only latest value matters; durability is pointless here.
- Driver offer concurrency: lock/lease the driver during an offer so two riders can't book the same driver.
- Surge pricing and ETA services are separable — mention them as independent consumers of the same Kafka location stream.

Jaydip

9. Web Crawler

Problem: Systematically download billions of web pages for a search index while being polite to websites. Tests queues, dedup, and distributed politeness.

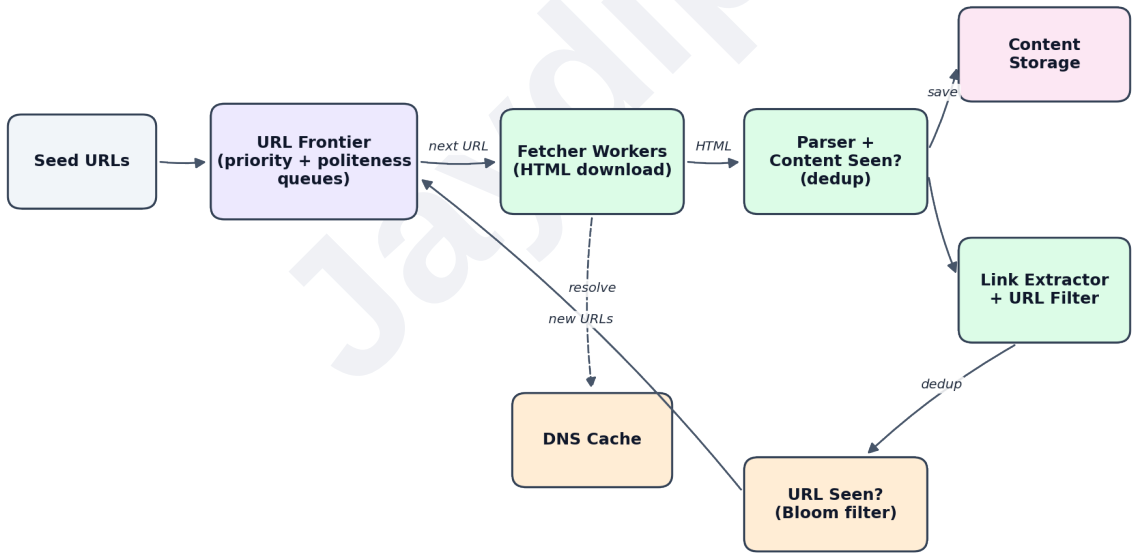
Requirements

- **Functional:** Start from seed URLs, download pages, extract links, keep crawling; store content for indexing; re-crawl based on freshness.
- **Non-functional:** Politeness (don't hammer one host; obey robots.txt), scalability (billions of pages), robustness (malformed HTML, spider traps, dead links), dedup of both URLs and content.

Back-of-the-envelope

- 1B pages/month ≈ 400 pages/s sustained; at ~100 KB/page ≈ 40 MB/s download bandwidth.
- Storage: 1B × 100 KB ≈ 100 TB/month for raw HTML (compressed less).
- URL-seen set: 5B URLs — a Bloom filter makes membership checks memory-cheap (small false-positive rate acceptable).

Architecture



Crawler loop: frontier → fetch → parse → dedup → extract links → back to frontier. Politeness lives in the frontier.

How it works

- The **URL Frontier** is the heart: front queues implement *priority* (PageRank, freshness), back queues implement *politeness* — one queue per host, each mapped to one worker, with per-host delay timers.
- Fetchers download pages (with a DNS cache — DNS lookups are a real bottleneck at this scale), respecting robots.txt (cached per host).
- 'Content seen?' dedups identical/near-identical pages via checksums (SimHash for near-dup); 'URL seen?' Bloom filter prevents re-queuing.
- Extracted links are normalized and filtered (blocklists, file types) before re-entering the frontier.

Key design decisions & trade-offs (deep-dive material)

- **Spider traps:** auto-generated infinite URL spaces (calendars, session IDs) — cap URL length and per-domain page budget.
- **BFS vs DFS:** BFS (FIFO-ish) with priority — DFS gets stuck deep in one site and is impolite.
- Freshness: re-crawl frequency proportional to observed change rate (news sites hourly, static pages monthly).
- Frontier is mostly on disk (billions of URLs) with in-memory buffers — say this so it doesn't sound like an infinite in-RAM queue.

Jaydip

10. Notification System

Problem: A shared platform any internal service can call to send push, email, and SMS notifications reliably — with retries, dedup, and user preferences.

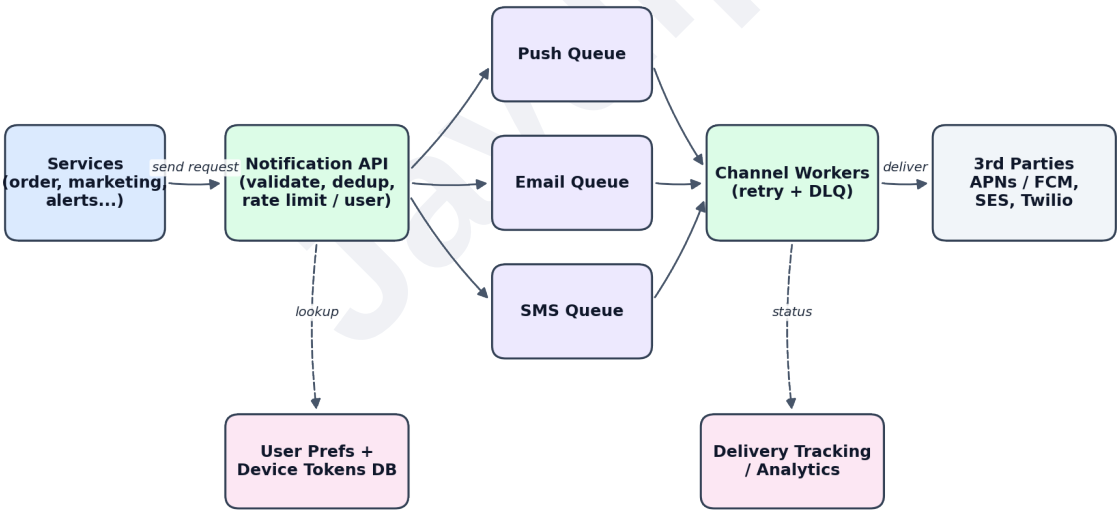
Requirements

- **Functional:** Send via push (iOS/Android), email, SMS; respect user opt-outs and quiet hours; templates; delivery tracking; scheduled/batch sends.
- **Non-functional:** At-least-once delivery (never silently drop), soft real-time (seconds OK), rate-limit per user (no spam), horizontal scale to millions/minute, third-party outages must not lose messages.

Back-of-the-envelope

- 100M notifications/day ≈ 1,200/s avg; marketing campaigns spike to 100K/s → queues absorb the burst.
- Third parties (FCM/APNs/Twilio/SES) throttle you → workers must apply provider-level rate limiting and backoff.

Architecture



Notification platform: one API, per-channel queues, workers with retry + DLQ, third-party providers do final delivery.

How it works

- Producer services call one Notification API → it validates, checks user preferences/opt-outs and device tokens, applies dedup + per-user rate caps → drops the message into the right channel queue.
- Channel workers pull, render templates, call the provider (APNs/FCM/SES/Twilio), and record status; failures retry with exponential backoff, then land in a dead-letter queue for inspection.
- Delivery receipts/webhooks from providers flow back into a tracking store for analytics and retry decisions.

Key design decisions & trade-offs (deep-dive material)

- **At-least-once + idempotency:** retries can duplicate → dedup key per notification (event_id) checked at send time; 'exactly once' is a myth worth saying out loud.
- **Queue per channel** (and per priority) isolates failures — an SMS provider outage shouldn't delay OTP push notifications.
- OTP/security messages get a dedicated high-priority lane with tighter SLAs than marketing.
- Token hygiene: prune invalid device tokens on provider feedback, or delivery rates rot silently.

Jaydip

11. Distributed Key-Value Cache

Problem: Design a Redis/Memcached-like distributed cache (or a Dynamo-style KV store) — get/put with low latency across many nodes. Tests partitioning, replication and consistency mechanics.

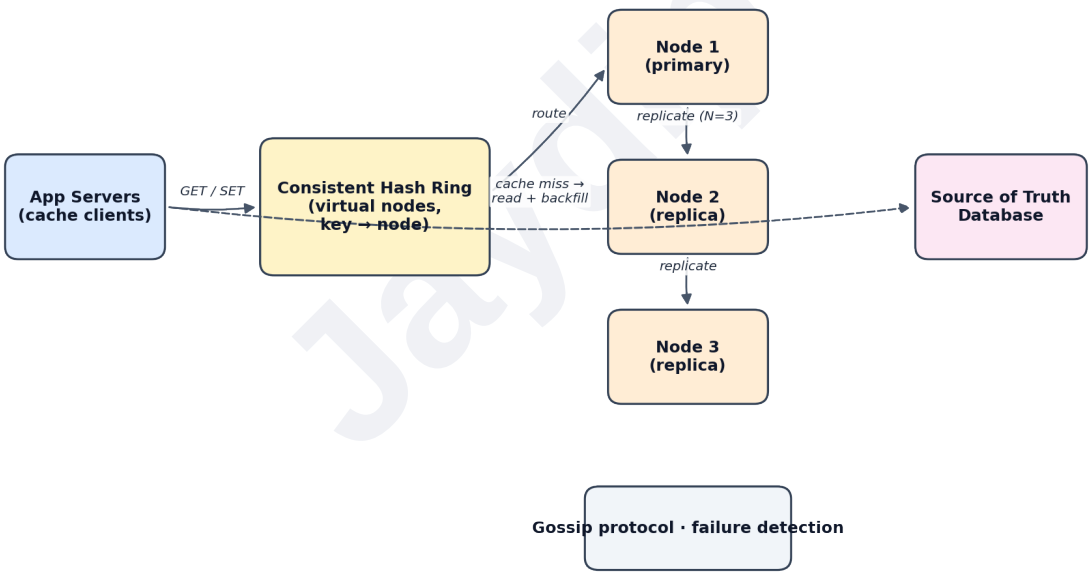
Requirements

- **Functional:** get(key), put(key, value), delete; TTL/eviction; scale by adding nodes with minimal data movement.
- **Non-functional:** Sub-millisecond reads, tunable consistency, no single point of failure, graceful behavior when nodes join/leave/fail.

Back-of-the-envelope

- Target 1M ops/s: a single Redis node does ~100K ops/s → ~10–20 nodes plus replicas.
- Cache 20% of a 10 TB working set ≈ 2 TB RAM → 32 nodes × 64 GB.

Architecture



Consistent hash ring routes each key to a primary; replication to N nodes; gossip detects failures.

How it works

- Clients (or a smart proxy) hash the key onto a **consistent hash ring**; virtual nodes smooth out load imbalance.
- Writes go to the key's primary and replicate to the next N-1 nodes on the ring (replication factor N=3 typical).
- **Quorums:** with N=3, W=2, R=2 → W+R > N gives read-your-writes; tune W/R for latency vs consistency.
- Node failures detected via gossip/heartbeats; *hinted handoff* stores writes for a downed node and replays on recovery; anti-entropy repair uses Merkle trees.

Key design decisions & trade-offs (deep-dive material)

- **Eviction:** LRU is the default; LFU resists one-off scans; TTL + lazy expiry keeps memory honest.

- **Cache problems interviewers love:** hot key (replicate that key / client-side cache), thundering herd on expiry (jittered TTL, single-flight lock), cache penetration for non-existent keys (negative caching / Bloom filter).
- **Conflict resolution** in AP mode: last-write-wins (simple, can lose data) vs vector clocks (Dynamo) — name both.
- Cache-aside vs write-through: cache-aside is simplest and most common; write-back is fastest but risks loss.

Jaydip

12. Payment System

Problem: Accept a payment at checkout, charge via a PSP (Stripe-like), keep money records correct forever. Tests correctness, idempotency, and reconciliation — consistency over availability.

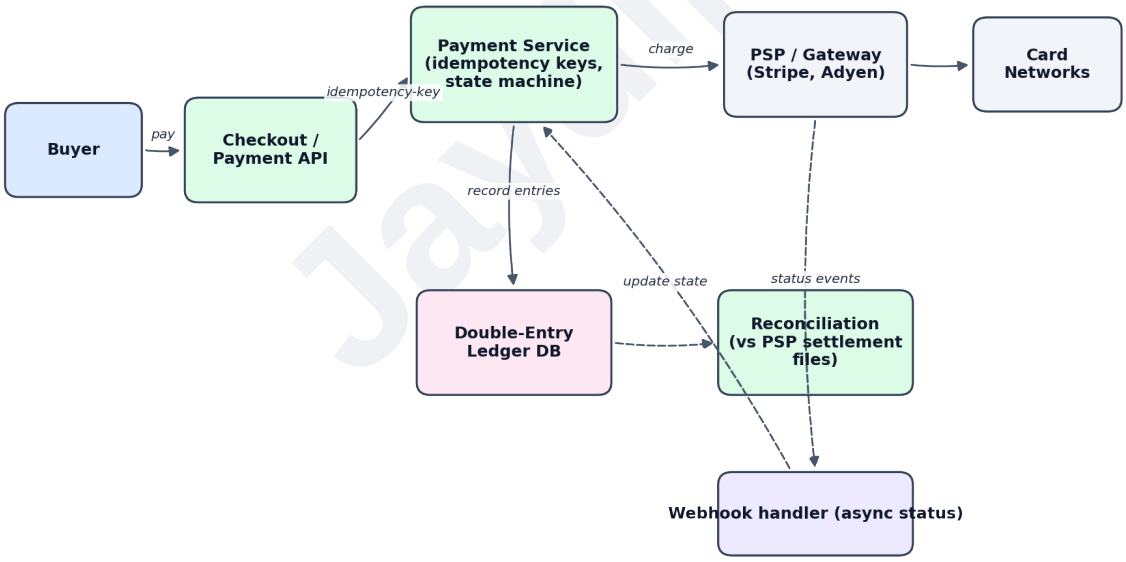
Requirements

- **Functional:** Pay-in (charge buyer), pay-out (settle seller), refunds; exactly-once *effect* per payment; auditable ledger.
- **Non-functional:** Strong consistency and durability trump latency; zero tolerance for double-charging or lost money; PCI scope minimized (never store raw card numbers — tokenize via PSP).

Back-of-the-envelope

- Even 1M payments/day ≈ 12 TPS — tiny traffic; the challenge is **correctness**, not scale. Say this in the interview.
- Every state change is recorded; ledger grows append-only forever (cheap, and required for audit).

Architecture



Payment flow: idempotent payment service drives a state machine, PSP handles cards, double-entry ledger records money movement, reconciliation catches drift.

How it works

- Checkout calls Payment Service with an **idempotency key** (retry-safe: same key → same result, never a second charge).
- Payment Service persists intent (state = PENDING) *before* calling the PSP, then charges via PSP; card data goes directly from the client to the PSP (tokenization) so your servers stay out of PCI scope.
- PSP responses and async webhooks drive the state machine (PENDING → SUCCEEDED/FAILED); webhooks must also be idempotent and verified.
- Every money movement is written as balanced **double-entry ledger** records (debit one account, credit another; sum = 0). Nightly **reconciliation** compares internal ledger vs PSP settlement files and flags mismatches.

Key design decisions & trade-offs (deep-dive material)

- **Why not distributed transactions with the PSP?** You can't 2PC with an external bank — instead: persist-then-call, timeouts treated as UNKNOWN (query PSP status, never blindly retry a charge without the idempotency key).
- **Exactly-once = at-least-once retries + idempotent operations** — the single most quotable line in this design.
- Ordering/consistency: a single payment's events must apply in order (per-payment partition key in Kafka).
- Failure drills: PSP down → queue and retry with backoff; webhook lost → poll PSP; DB commit fails after PSP success → reconciliation catches it. Walk one of these unprompted.

Jaydip

13. Search Autocomplete (Typeahead)

Problem: As the user types, suggest the top-k most popular completions within ~100 ms per keystroke.

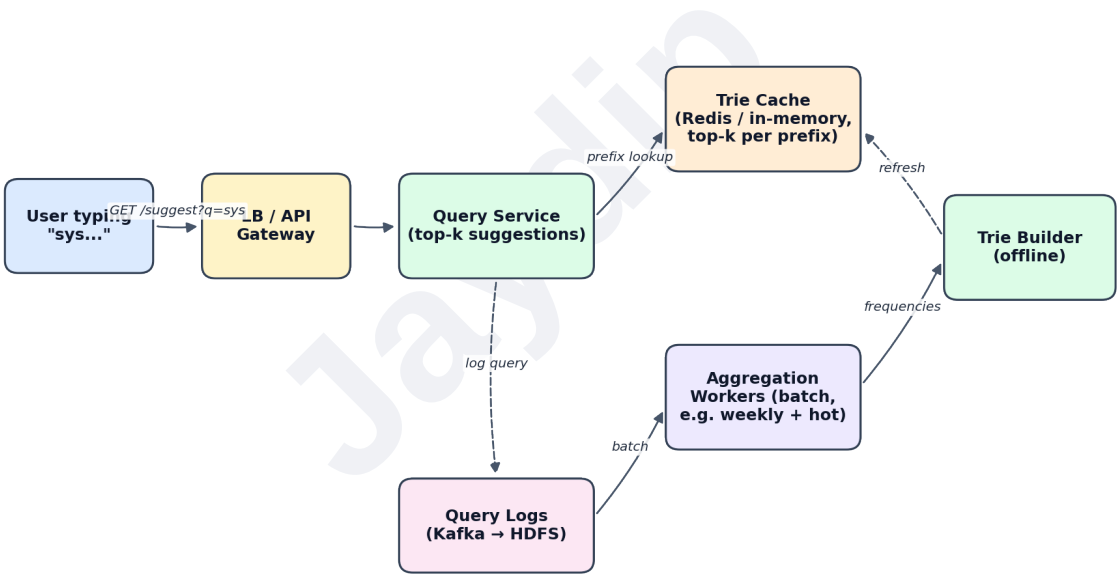
Requirements

- **Functional:** Return top 5–10 suggestions per prefix; ranked by popularity (and optionally freshness/personalization); update rankings from real query logs.
- **Non-functional:** p99 < 100 ms per keystroke (feels instant), high availability, suggestions may be minutes-to-hours stale — a great example of trading freshness for speed.

Back-of-the-envelope

- 10M queries/day, avg 4 keystrokes each hitting the service (after client debounce) ≈ 500–2,000 QPS.
- Trie with top-k precomputed *per node*: lookup = O(prefix length), not O(dataset).
- English prefix space is small enough to shard the trie by first character(s) and hold in RAM.

Architecture



Typeahead: reads served entirely from a precomputed trie cache; query logs feed an offline pipeline that rebuilds rankings.

How it works

- **Read path:** keystroke (debounced ~150 ms client-side) → Query Service → trie cache lookup for the prefix → return precomputed top-k. No DB in the hot path.
- **Write/analytics path:** queries logged to Kafka → aggregated in batch (e.g., weekly base + hourly 'trending' delta) → Trie Builder constructs a new trie → atomically swap into serving cache.
- At each trie node, store the top-k completions directly — turning a subtree walk into a single lookup.

Key design decisions & trade-offs (deep-dive material)

- **Why precompute top-k per node:** walking a subtree per keystroke at p99 < 100 ms is impossible; memory-for-latency is the core trade.
- Update strategy: real-time trie updates are complex; batch rebuild + hot-topic overlay is the pragmatic answer.

- Sharding: by prefix range with load-aware splits (naive a–z sharding overloads 's' and 't').
- Extras worth naming: filter offensive terms at build time, personalize by re-ranking the top 20 client-side.

Jaydip

14. Distributed Job Scheduler

Problem: A cron-as-a-service: users register one-off or recurring jobs; the system executes them on time, at scale, exactly-once-ish. Increasingly common in interviews.

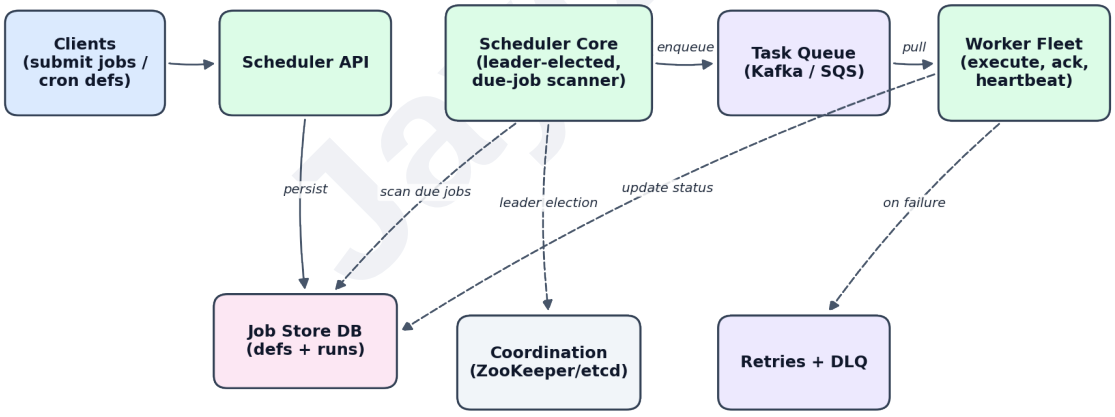
Requirements

- **Functional:** Create/update/delete jobs (cron expressions or run-at timestamps), execute within seconds of due time, retries with backoff, job status/history API.
- **Non-functional:** No missed jobs (durability of schedules), no duplicate execution of the same run, horizontal scale to millions of jobs, scheduler itself must be HA.

Back-of-the-envelope

- 10M jobs, avg hourly → ~2,800 executions/s; a partitioned due-time index handles this easily.
- Job store: index by (next_run_time) so 'what's due in the next minute' is a cheap range scan, not a full scan.

Architecture



Scheduler: leader-elected core scans due jobs into a queue; stateless workers execute with heartbeats, retries and a DLQ.

How it works

- Jobs persist in a DB indexed by next_run_time (sharded by time bucket + job hash at scale).
- The Scheduler Core — **leader-elected** via ZooKeeper/etcd so only one scans at a time (per shard) — polls for due jobs every second and enqueues them to Kafka/SQS.
- Stateless workers pull tasks, take a **lease** (visibility timeout), execute, heartbeat while running, and ack on success; missed heartbeats → task re-queued.
- On completion, workers compute the next run (for cron jobs), update the store, and write run history. Failures retry with exponential backoff → DLQ after N attempts.

Key design decisions & trade-offs (deep-dive material)

- **Exactly-once execution:** impossible in general — the answer is at-least-once delivery + idempotent job handlers (or a 'run_id claimed' row with a unique constraint).
- **Why leader election:** two schedulers scanning the same shard → duplicate enqueues; alternatively use DB row-level locking (SELECT ... FOR UPDATE SKIP LOCKED) — name both.
- Clock skew: schedule on DB time, not worker wall clocks; tolerate ± a few seconds.
- Thundering herd at popular times (00:00 cron): add jitter to schedules and rate-limit dequeue.

Jaydip

15. Rapid-Fire: Trade-offs Interviewers Love to Probe

Question	The answer they want
SQL vs NoSQL?	SQL: transactions, joins, moderate scale. NoSQL: massive write scale, flexible schema, simple a
Strong vs eventual consistency?	Money/inventory → strong (CP). Feeds/likes/counters → eventual (AP). Tie it to user impact of s
Push vs pull (fanout)?	Push for normal users (fast reads), pull for celebrities (cheap writes), hybrid overall.
Vertical vs horizontal scaling?	Vertical first (simple), horizontal for real scale + fault tolerance; horizontal forces statelessness.
Cache invalidation?	TTL + explicit invalidation on writes; jitter TTLs; accept brief staleness where it's harmless.
How to shard?	Pick a key with even distribution matching your top query (user_id, conversation_id). Beware hot
Sync vs async processing?	Anything not needed for the user's immediate response goes to a queue. Queues absorb spikes
Exactly-once delivery?	Doesn't exist end-to-end. At-least-once + idempotent consumers = effectively exactly-once.
Monolith vs microservices?	Start monolith; split on team/scaling boundaries. Microservices trade dev speed for operational c
How do you handle a single point of failure?	Redundancy + health checks + automatic failover; then ask 'what about the failover coordinator?

Final checklist before any interview

- Rehearse the 4-step framework until it's automatic.
- Be able to draw each diagram in this handbook from memory in under 3 minutes.
- For every component you draw, have one trade-off sentence ready.
- Practice out loud — system design is a communication exam wearing an engineering costume.

Jaydip